Part 4 – LLM-Powered Feature

Project Overview

This project extends the Used Car Price Classification system developed in Part 3 by integrating a Large Language Model (LLM) to generate clear and structured explanations for machine learning predictions. Along with making predictions, the system now explains why a vehicle was classified into a particular price category. The implementation also includes prompt engineering, JSON output validation, temperature comparison, and privacy guardrails to ensure reliable and secure responses.

Chosen Track: Track C – Model Prediction Explanation Pipeline

Objective

The main objective of this project is to combine a trained machine learning model with a Large Language Model to provide understandable explanations for model predictions. After the trained model predicts the price category and its confidence score, the prediction is sent to the LLM, which generates a structured explanation in JSON format. This makes the prediction process more transparent and easier to interpret.

Dataset

Dataset: Used Car Price Dataset

Target Variable

Price Category

- **0** – Below Median Price
- **1** – Above Median Price

Features Used

- Brand
- Model
- Model Year
- Mileage
- Fuel Type
- Engine
- Transmission
- Exterior Color
- Interior Color
- Accident History
- Clean Title

Machine Learning Model

The model developed in Part 3 was reused for this task.

Selected Model: Random Forest Classifier Pipeline

Pipeline Components

- SimpleImputer (strategy='median')
- StandardScaler
- RandomForestClassifier

The trained pipeline was loaded using:

joblib.load("best_model.pkl")

This allowed predictions to be generated without retraining the model.

LLM Integration

The prediction explanation was generated using an external Large Language Model accessed through the Python **requests** library.

To improve security, the API key is stored as an environment variable rather than being written directly into the code.

os.environ["LLM_API_KEY"]

This approach helps protect sensitive credentials and follows good development practices.

System Prompt

The following system prompt was used to ensure that the LLM always returns a structured JSON response.

You are an AI assistant that explains machine learning predictions.

Return ONLY valid JSON.

{

"prediction_label":"",

"confidence_level":"",

"top_reason":"",

"second_reason":"",

"next_step":""

}

User Prompt Template

Each prediction is converted into the following prompt before being sent to the LLM.

Feature Values:

{feature_values}

Predicted Class:

{prediction}

Predicted Probability:

{probability}

Explain the prediction and return ONLY valid JSON.

Providing the prediction, probability, and feature values helps the model generate explanations that are relevant to each vehicle.

Why Temperature = 0?

The temperature was set to **0** to produce consistent and repeatable responses. Since the task requires structured JSON output rather than creative text, using a temperature of zero reduces randomness and ensures the same input produces the same explanation every time.

Higher temperature values can generate more varied wording, but they may also introduce inconsistencies in formatting or content, making validation more difficult.

JSON Output Schema

Each response generated by the LLM must include the following fields.

Field	Type
prediction_label	String
confidence_level	String
top_reason	String
second_reason	String
next_step	String

The generated response is validated using the **jsonschema** package. If the response is not valid JSON or does not match the required schema, the system returns a fallback response instead of invalid data.

PII Guardrail

Before sending any request to the LLM, a regular expression (regex) is used to check whether the input contains personally identifiable information (PII).

The guardrail detects:

- Email addresses
- Phone numbers

If any sensitive information is found, the request is blocked and is not sent to the API.

Example

Test Input	**Result**
student@example.com	Blocked
Toyota Camry 2021 with 45,000 km	Allowed

This simple privacy check helps prevent accidental sharing of personal information.

Example JSON Response

{

"prediction_label": "Above Median Price",

"confidence_level": "High",

"top_reason": "The vehicle has a newer model year and lower mileage.",

"second_reason": "The vehicle has a clean accident history.",

"next_step": "Consider this vehicle a strong purchase candidate."

}

Response Validation

Every response returned by the LLM follows the validation process below:

1. Receive the raw response from the LLM.
2. Remove any unnecessary whitespace.
3. Parse the response using json.loads().
4. Validate it against the predefined JSON schema.
5. If validation fails, return a predefined fallback response.

This ensures that only valid and properly formatted JSON is used by the application.

Conclusion

This project successfully combines a trained Random Forest classifier with a Large Language Model to provide structured explanations for machine learning predictions. The addition of JSON schema validation ensures that responses remain consistent and machine-readable, while the PII guardrail helps protect sensitive information before any API request is made. Using a temperature setting of **0** produces deterministic and reliable outputs, making the system suitable for explainable AI applications and real-world deployment. Overall, the project demonstrates how machine learning and LLMs can work together to build prediction systems that are not only accurate but also transparent and easier for users to understand.